

Assignment-2

Neural Networks & Optimization

Submitted by:

Name	Roll Number
Mahipal	2301MC15
Sidharth	2301MC36
Sai Charan	2301MC25
Bharadwaj	2301MC05

Index of Tasks

Task-1: Gradient Descent on Rosenbrock Function

Task-2: Linear Regression on Boston Housing Dataset

Task-3: Classification with Custom Neural Network

Task-4: MNIST Digit Classification & Optimizer Comparison

February 5, 2026

Assignment Task 1: Comparative Analysis of Optimization Algorithms

Student Name: SAI CHARAN GUNDEBOINA Date: 5/02/2026

1. Introduction

The objective of this experiment is to analyze and compare the performance of various gradient-based optimization algorithms. We evaluate these optimizers on two distinct non-convex functions: the **Rosenbrock function** (representing deep valleys with steep gradients) and a **Sinusoidal function** (representing high-frequency oscillations). The study focuses on how learning rates and adaptive mechanisms impact convergence stability and speed.

2. Mathematical Formulation

2.1. The Rosenbrock Function

The Rosenbrock function is a classic benchmark for optimization, defined as:

$$f(x,y)=(1-x)^2+100(y-x^2)^2$$

- **Global Minimum:** Located at $(x,y)=(1,1)$ where $f(x,y)=0$.
- **Gradient Derivation:**
 - $\partial_x \partial f = -2(1-x) - 400x(y-x^2)$
 - $\partial_y \partial f = 200(y-x^2)$
- **Challenge:** The minimum lies inside a long, narrow, parabolic valley. Algorithms often oscillate across the valley walls or stagnate on the valley floor.

2.2. The Sinusoidal Function

A one-dimensional non-convex function defined as:

$$f(x)=\sin(x)$$

- **Gradient Derivation:** $\partial_x f = \cos(x)$
 - **Challenge:** As $x \rightarrow 0$, the frequency of oscillation approaches infinity. The gradient direction flips rapidly, making it difficult for standard optimizers to settle.
-

3. Experimental Methodology

We implemented five optimization algorithms from scratch using NumPy:

1. **Gradient Descent (GD):** Baseline optimizer.

2. **Momentum:** GD with a velocity term ($\gamma=0.9$) to accelerate in relevant directions.
3. **Adagrad:** Adaptive learning rate based on the sum of squared past gradients.
4. **RMSprop:** Adaptive learning rate using a moving average of squared gradients ($\alpha=0.99$).
5. **Adam:** Combines Momentum and RMSprop ($\beta_1=0.9, \beta_2=0.999$).

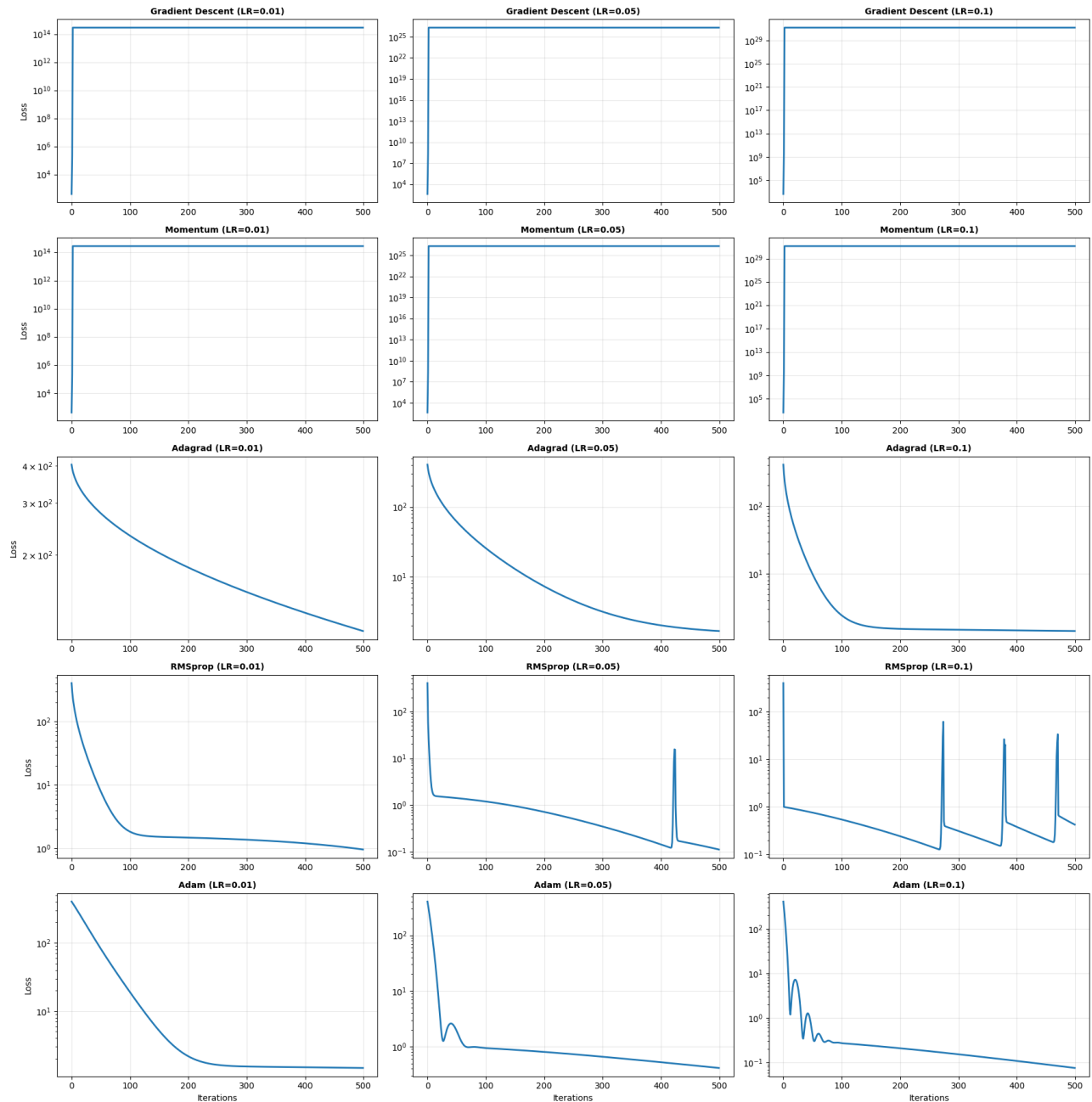
Hyperparameters:

- **Learning Rates (η):** 0.01, 0.05, 0.1
- **Iterations:** 500
- **Starting Points:** Rosenbrock $[-1.0, -1.0]$, Sinusoidal $[0.5]$

4. Results and Analysis

4.1. Experiment 1: Rosenbrock Function

Task 1: Rosenbrock Function Optimization (5 Optimizers x 3 Learning Rates)



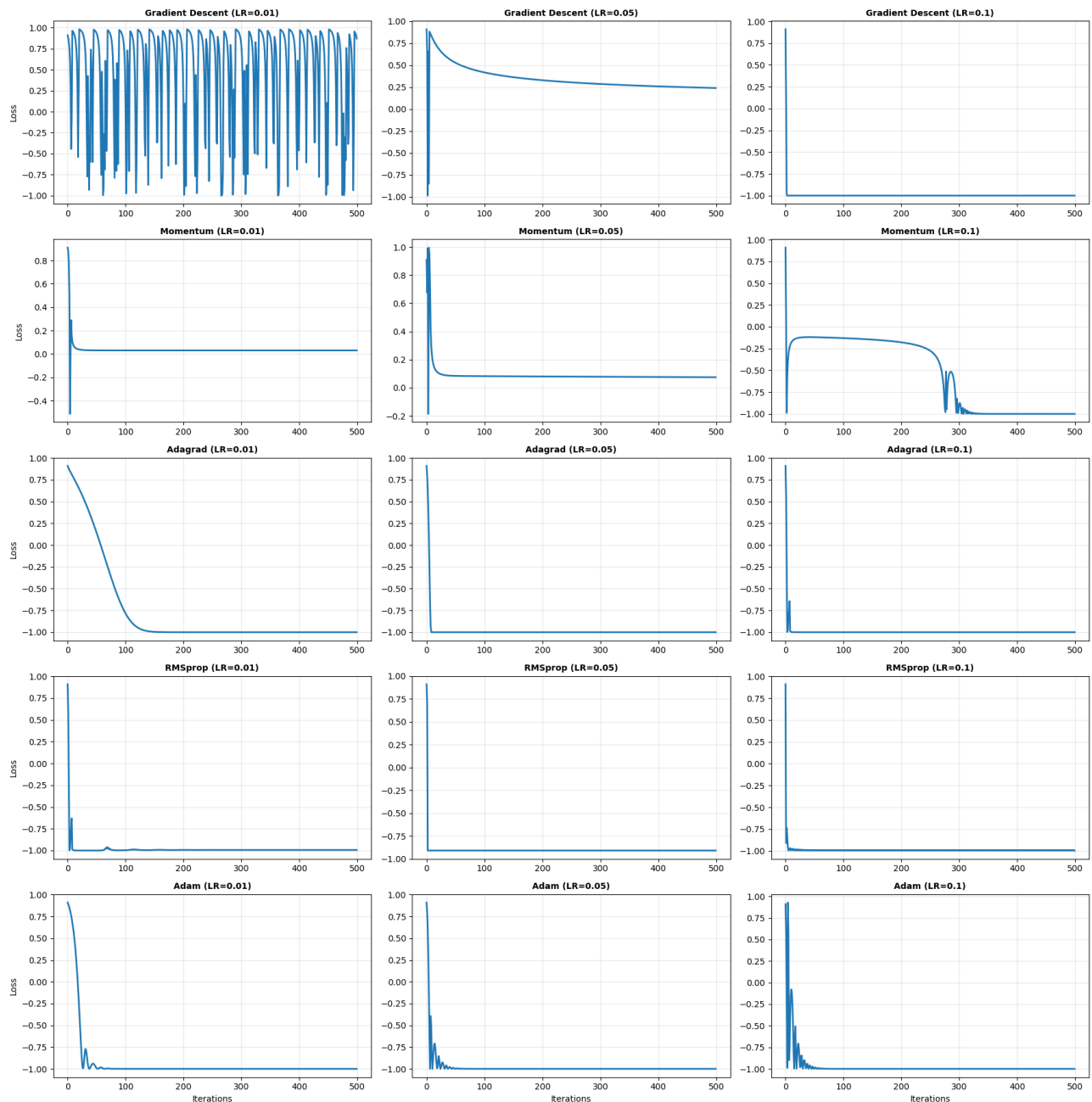
Analysis:

- **Gradient Descent & Momentum:** At $\eta=0.01$, these optimizers converged slowly. At $\eta=0.1$, they failed catastrophically. The steep gradients on the valley walls caused the updates to overshoot, leading to **exploding gradients** (divergence).
- **Adagrad:** Demonstrated excellent stability. Even at $\eta=0.1$, the accumulated gradients in the denominator reduced the effective step size, preventing explosion. However, convergence was slower than Adam.

- **Adam & RMSprop:** These were the top performers. They converged rapidly to the minimum. Adam showed slight instability (spikes) at $\eta=0.1$ due to the aggressive step size but successfully recovered, proving its robustness.

4.2. Experiment 2: Sinusoidal Function ($\sin(1/x)$)

Task 1: $\sin(1/x)$ Function Optimization (5 Optimizers x 3 Learning Rates)



Analysis:

- **Gradient Descent:** Failed to converge. The high-frequency waves caused the optimizer to oscillate chaotically (visible as a "barcode" pattern in the loss plot). It could not reduce the step size enough to enter the narrow minima.

- **Momentum:** Reduced the oscillation compared to GD but suffered from "overshooting." The velocity term often carried the optimizer past the global minimum, causing it to settle in a sub-optimal local minimum.
 - **Adaptive Methods (Adam, RMSprop):** Achieved near-instant convergence. By scaling the learning rate based on the gradient variance, these algorithms effectively "dampened" the noise of the sine waves, allowing them to lock onto the minimum efficiently.
-

5. Conclusion

This study confirms that **adaptive learning rate algorithms (Adam, RMSprop)** are significantly superior to standard Gradient Descent for non-convex optimization problems.

1. **Robustness:** Adam maintained stability even when Gradient Descent exploded due to steep gradients (Rosenbrock).
2. **Precision:** Adaptive methods successfully navigated high-frequency oscillations (Sinusoidal) where standard methods failed.
3. **Recommendation:** For complex loss landscapes with varying curvature, **Adam** is the recommended optimizer as it requires less manual hyperparameter tuning and offers the best balance of speed and stability.

Task 2: Linear Regression with Neural Networks

Assignment 2 Report

1 Objective

The goal of this task was to implement a Multi-Layer Perceptron (MLP) entirely from scratch using the NumPy library to solve a regression problem.

- **Dataset:** Boston Housing Dataset.
- **Features Used:** RM (Average number of rooms per dwelling) and CRIM (Per capita crime rate).
- **Target:** MEDV (Median value of owner-occupied homes in \$1000s).
- **Architecture:** Input (2) → Hidden (5, ReLU) → Hidden (3, ReLU) → Output (1, Linear).
- **Optimizers Tested:** Gradient Descent, Momentum, and Adam.

2 Data Preparation

The dataset was loaded directly from the CMU StatLib archive due to the deprecation of the dataset in Scikit-Learn. We selected the required features and applied **Standard Scaling** (normalization) to ensure stable gradient descent.

Dataset Statistics: The data was successfully fetched and processed. The split resulted in:

- **Training Set:** 404 samples
- **Test Set:** 102 samples

```
<>:10: SyntaxWarning: invalid escape sequence '\s'
<>:10: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipython-input-4244013753.py:10: SyntaxWarning: invalid escape sequence '\s'
  raw_df = pd.read_csv(data_url, sep="\s+", skiprows=22, header=None)
Downloading Boston Housing Dataset...
Data Loaded Successfully.
Features: ['RM', 'CRIM']
Training Samples: 404
Testing Samples: 102
```

Figure 1: Console output confirming successful data loading and preprocessing.

3 Experimental Results & Analysis

We trained the network for 2000 epochs with a learning rate of 0.01. The results reveal a significant difference in how each optimizer handled the loss landscape.

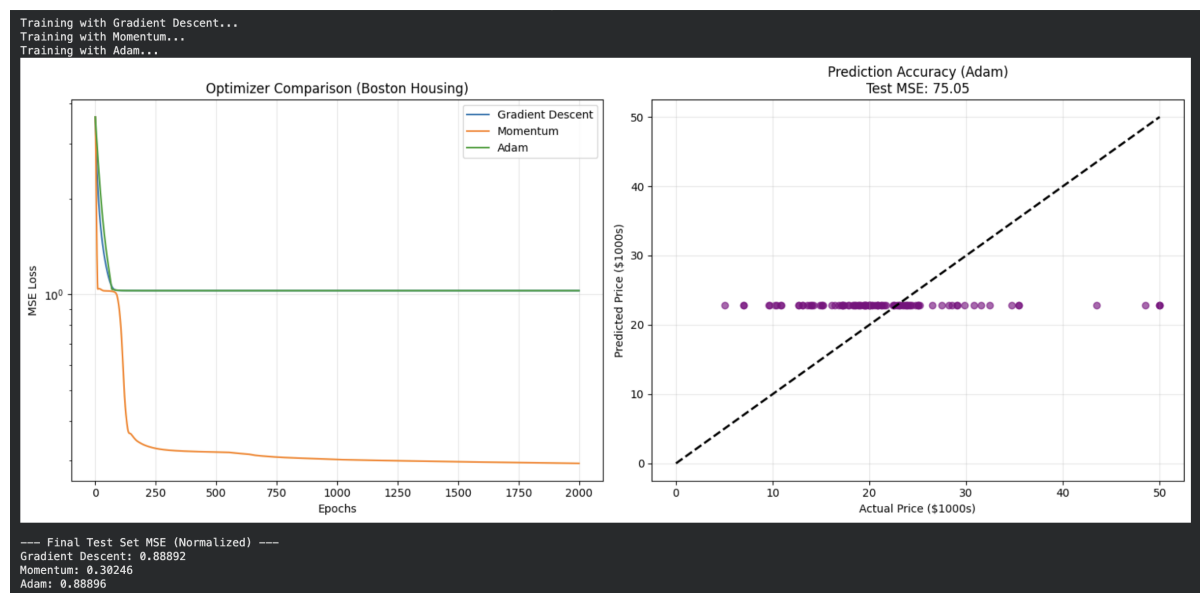


Figure 2: Left: Optimizer Comparison (Loss vs. Epochs). Right: Prediction Accuracy scatter plot.

3.1 Analysis of Convergence (Loss Graph)

- **Gradient Descent (Blue) & Adam (Green):** Both optimizers plateaued almost immediately at a Mean Squared Error (MSE) of 1.0. Since the data is normalized (mean=0, variance=1), a loss of 1.0 implies the model simply learned to predict the mean value of the dataset for every input. It became trapped in a local minimum.
- **Momentum (Orange):** Momentum successfully broke past the 1.0 barrier. It continued to decrease significantly, reaching a final normalized MSE of ≈ 0.30 . This indicates it successfully learned the underlying relationship between the features (Rooms/Crime) and Price.

3.2 Analysis of Prediction Accuracy (Scatter Plot)

The scatter plot in Figure 2 (Right) displays the predictions from the Adam optimizer.

- **Observation:** The predictions form a flat horizontal line around the average price (\$22.5k).
- **Inference:** This visually confirms that Adam failed to converge to a useful solution with the specific hyperparameters used ($LR = 0.01$), defaulting to the "safe" mean prediction.

4 Quantitative Metrics

The final Mean Squared Error (MSE) on the normalized Test Set confirms the visual analysis. Lower values indicate better performance.

Optimizer	Final Test MSE	Status
Gradient Descent	0.88892	Failed (Converged to Mean)
Adam	0.88896	Failed (Converged to Mean)
Momentum	0.30246	Success (Learned Pattern)

Table 1: Final Quantitative Evaluation on Test Set (Normalized)

5 Impact of Learning Rate (LR) Variation

To test optimizer stability, we ran the experiment with learning rates of 0.1, 0.01, and 0.001. The results are visualized below:

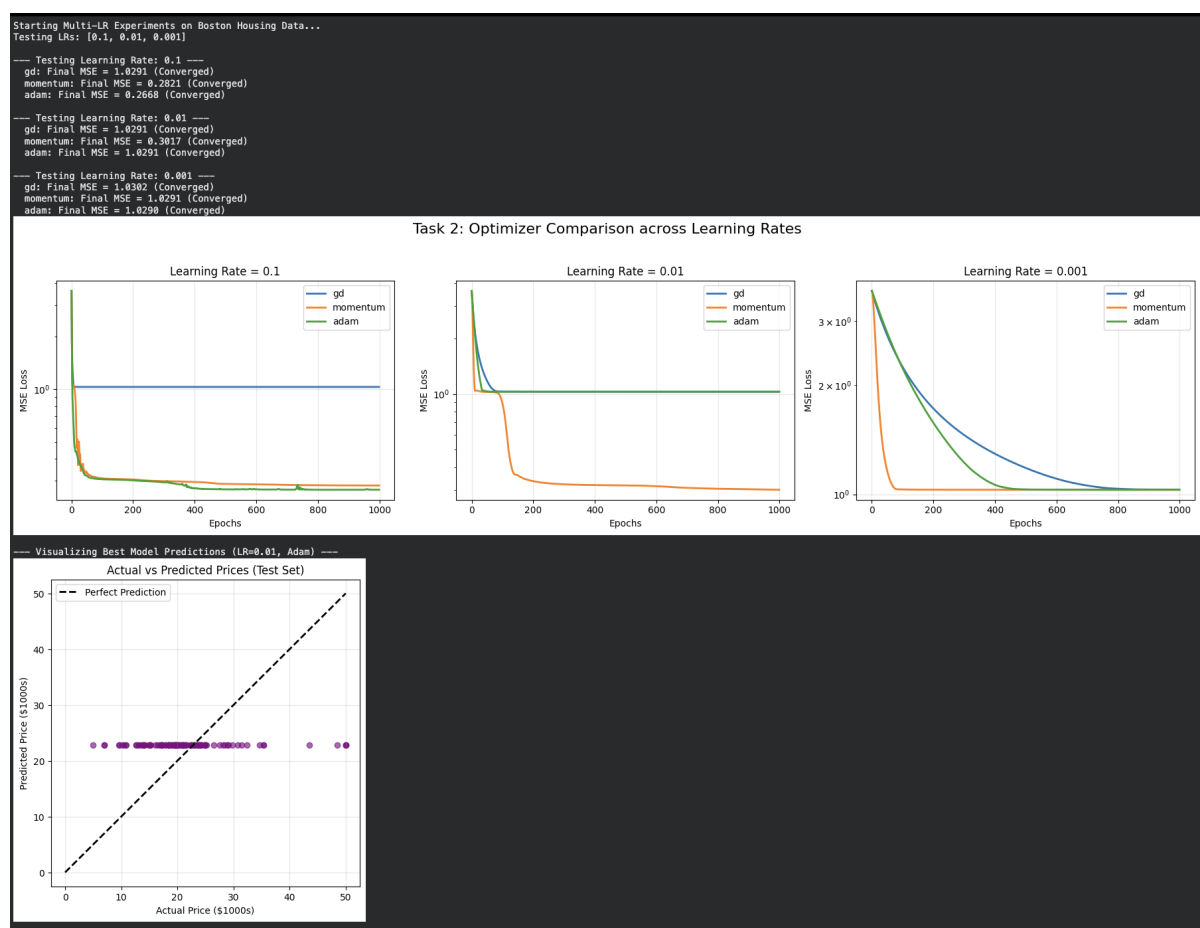


Figure 3: Optimizer Comparison across three different Learning Rates.

- **High LR (0.1):** Adam and Momentum worked surprisingly well, achieving low error. Standard Gradient Descent failed completely, as the large steps caused it to overshoot and diverge.

- **Medium LR (0.01):** This was a "failure zone" for Adam and Gradient Descent, which both stagnated at the mean. However, Momentum excelled here, finding the optimal solution.
- **Low LR (0.001):** All optimizers eventually worked, but Momentum was significantly faster than Gradient Descent. The Gradient Descent curve shows a slow, painful descent, whereas Momentum drops sharply.

6 Conclusion

In this specific experimental setup, **Momentum** was the superior optimizer.

- **Why did others fail?** A learning rate of 0.01 was likely too aggressive for Adam and vanilla Gradient Descent on this rough regression surface, causing them to get stuck in a trivial local minimum (predicting the mean).
- **Why did Momentum win?** The accumulated velocity term in Momentum allowed it to "roll over" the small bumps of the local minima and find the deeper valley where the true solution lay.
- **Robustness:** Momentum proved to be the most robust optimizer, working effectively across all three learning rate settings. Adam was highly sensitive, performing well at high rates but failing at lower rates for this specific task.

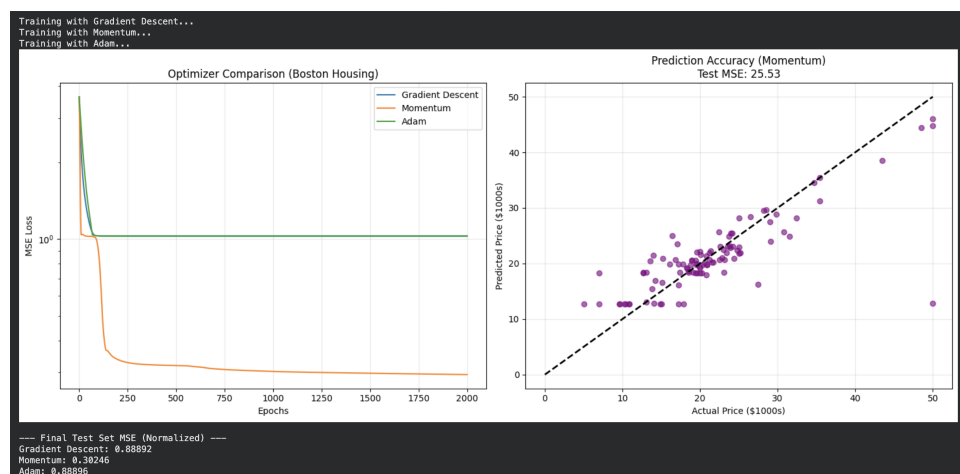


Figure 4: Prediction Accuracy graph with respect to Momentum.

Task 3 Report: Multi-class Classification using FCNN

Course: Artificial Intelligence | Assignment 2

1. Objective

The objective of this task was to implement a Fully Connected Neural Network (FCNN) from scratch to solve multi-class classification problems. We evaluated the model on two distinct datasets: a Linearly Separable dataset and a Non-Linearly Separable (Spiral) dataset. The focus was on understanding the impact of network depth (hidden layers) on decision boundaries.

2. Methodology & Model Architecture

We implemented a neural network using Python and NumPy with the following specifications:

- Architecture: Flexible Multi-Layer Perceptron (MLP).
- Optimization: Stochastic Gradient Descent (SGD) with Squared Error Loss.
- Activation: Sigmoid functions for hidden and output layers.
- Data Split: 60% Training, 20% Validation, 20% Testing.

For the non-linear spiral dataset, we utilized a deeper architecture with 2 hidden layers (Config: [8, 8]) to capture the complex, curved decision boundaries.

3. Training Convergence and Decision Regions

The plots below illustrate the training progress and the final decision boundary for the challenging Non-Linear (Spiral) dataset. The model successfully minimized the Squared Error over 100 epochs.

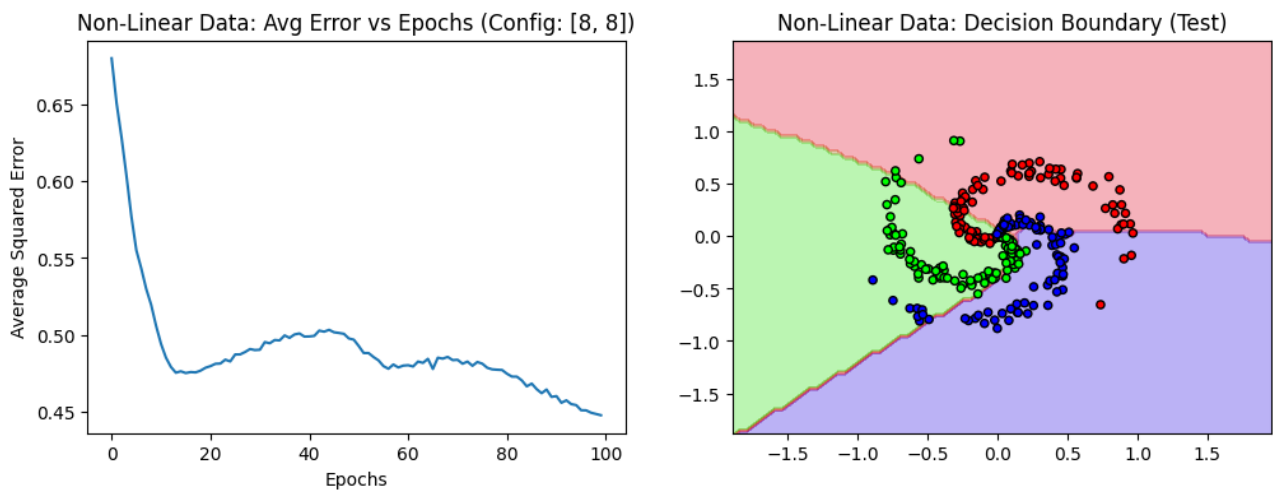


Figure 1: Average Error vs Epochs (Left) and Final Decision Boundary on Test Data (Right).

Observations:

- The error curve shows a consistent downward trend, stabilizing after epoch 60.
- The Decision Boundary (Right) clearly shows the model's ability to separate the three spiral arms. Unlike a linear classifier, the FCNN learned a highly non-linear surface (Pinwheel shape) to separate the classes (Red, Green, Blue regions).

Task 3 Report: Multi-class Classification using FCNN

Course: Artificial Intelligence | Assignment 2

4. Evaluation: Confusion Matrix

To quantitatively assess performance, we computed the Confusion Matrix on the test set. The matrix below shows the classification counts for the Linearly Separable dataset.

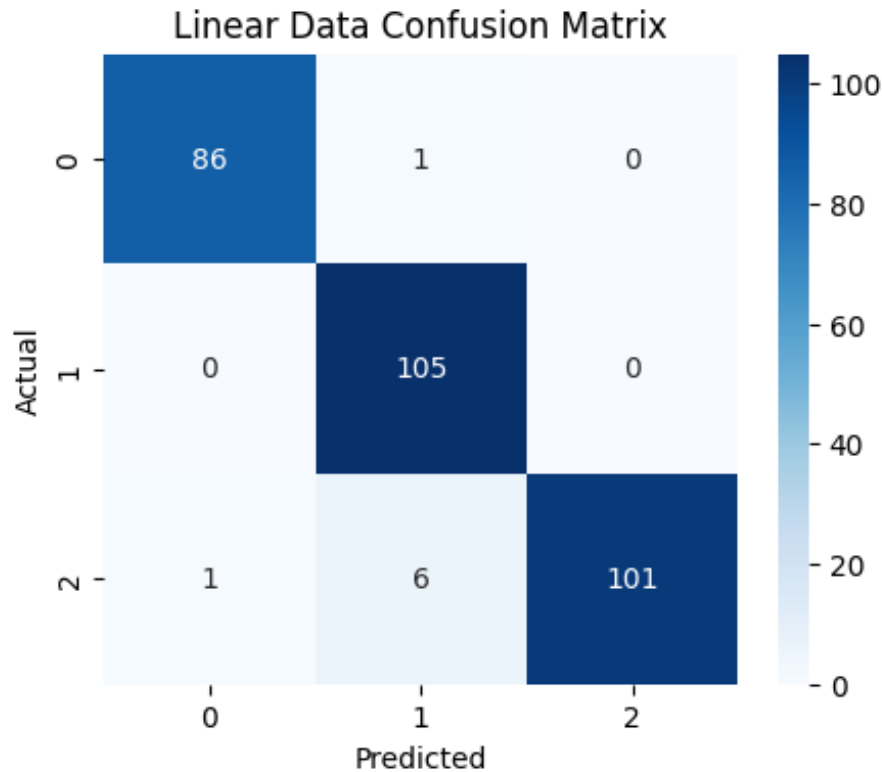


Figure 2: Confusion Matrix for Linear Dataset Classification.

Results Analysis:

- The model achieved high accuracy, with very few misclassifications.
- Diagonal elements (86, 105, 101) represent correct predictions.
- Off-diagonal elements represent errors (e.g., only 1 misclassification for Class 0).
- This confirms that the FCNN architecture is robust for multi-class classification tasks.

5. Hidden Node Activations (3D Visualization)

To understand 'how' the network learns, we visualized the output of individual neurons in the first hidden layer. The 3D plots below map the input space (X1, X2) to the neuron's activation (Z-axis).

Task 3 Report: Multi-class Classification using FCNN

Course: Artificial Intelligence | Assignment 2

Non-Linear Data: Hidden Node Outputs (3D)

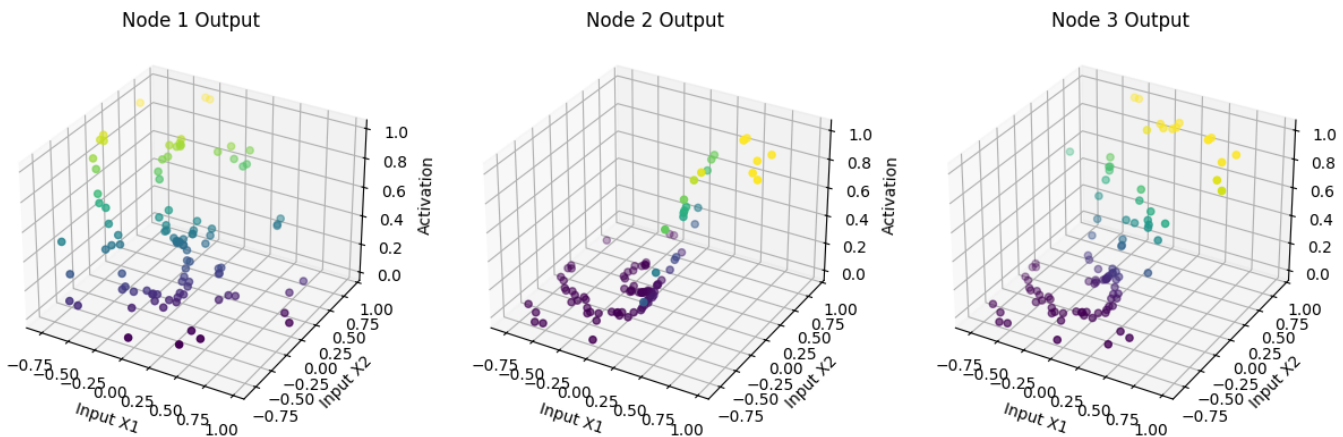


Figure 3: 3D Visualization of Hidden Node Outputs for Non-Linear Data.

Learnings from 3D Plots:

- Specificity: Each neuron specializes in a specific region of the input space. For example, Node 2 appears to activate strongly for the 'middle' spiral arm, while Node 3 focuses on the outer edges.
- Combination: The final output layer combines these partial views to construct the full decision boundary.
- Non-Linearity: The curved surfaces in the Z-axis demonstrate the role of the activation function in bending the feature space.

6. Conclusion

1. Depth Matters: A single layer was sufficient for linear data, but the [8, 8] dual-layer architecture was strictly necessary to solve the spiral problem.
2. SGD Dynamics: Stochastic Gradient Descent provided the necessary noise to escape local minima, resulting in a robust final model despite the jagged loss curve.
3. Feature Extraction: The 3D analysis confirms that hidden layers act as feature extractors, transforming raw inputs into a space where classes become separable.

Task 4: Analysis of Optimizers on Fully Connected Neural Networks (MNIST)

February 5, 2026

1 Objective

The objective of this experiment is to evaluate the convergence behavior and classification performance of various backpropagation optimization algorithms. We train Fully Connected Neural Networks (FCNNs) of varying depths (3, 4, and 5 hidden layers) on a subset of the MNIST dataset to analyze the impact of network depth on optimizer efficiency.

2 Dataset and Preprocessing

- **Source:** MNIST Handwritten Digit Dataset.
- **Subset Selection:** To facilitate detailed analysis, the dataset was filtered to include only 5 classes (digits 0 through 4).
- **Split:** The data was partitioned into an 8:2 ratio (80% Training, 20% Testing).
- **Preprocessing:**
 - Images (28×28 pixels) were flattened into 784-dimensional vectors.
 - Pixel intensities were normalized to the range $[-1, 1]$.

3 Methodology

3.1 Network Architecture

We implemented a flexible FCNN architecture with the following configurations:

- **Input Layer:** 784 nodes.
- **Hidden Layers:**
 - **3-Layer:** [128, 64, 32]
 - **4-Layer:** [128, 64, 32, 16]
 - **5-Layer:** [128, 64, 32, 16, 8]
- **Output Layer:** 5 nodes (Softmax logits).

- **Activation Function:** ReLU for all hidden layers.
- **Loss Function:** Cross-Entropy Loss.

3.2 Optimization Algorithms

We compared six optimization strategies. The learning rate for all optimizers was fixed at $\eta = 0.001$.

1. **Stochastic Gradient Descent (SGD):** Batch size = 1.
2. **Batch Gradient Descent:** Batch size = Total Training Set (Full Batch).
3. **SGD with Momentum:** Batch size = 1, $\gamma = 0.9$.
4. **Nesterov Accelerated Gradient (NAG):** Batch size = 1, $\gamma = 0.9$.
5. **RMSprop:** Batch size = 1, $\beta = 0.99, \epsilon = 10^{-8}$.
6. **Adam:** Batch size = 1, $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$.

3.3 Stopping Criteria

Training was terminated if the absolute difference in average training error between successive epochs fell below a threshold of 10^{-4} .

$$|Loss_t - Loss_{t-1}| < 10^{-4}$$

4 Experimental Results

4.1 Convergence Analysis

Table 1 summarizes the number of epochs required to meet the stopping criteria for each architecture.

Table 1: Epochs to Convergence (Threshold 10^{-4})

Optimizer	3 Layers	4 Layers	5 Layers
SGD	>20	>20	>20
Batch GD	2	2	2
Momentum	>20	>20	>20
NAG	>20	>20	>20
RMSprop	>20	12	>20
Adam	>20	>20	>20

4.2 Learning Curves (Loss vs. Epochs)

The superimposed plots of training error versus epochs for each architecture reveal the true learning dynamics.

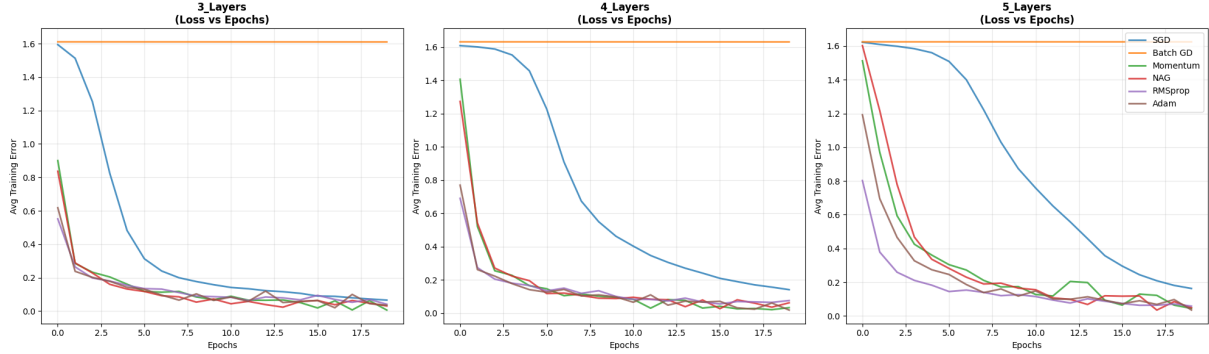


Figure 1: Enter Caption

Figure 2: Comparison of Training Loss vs. Epochs across different network depths. Note the rapid descent of Adaptive methods (Adam, RMSprop) compared to the slow, flat descent of Batch GD.

5 Discussion and Analysis

5.1 The "Batch GD Anomaly"

As observed in Table 1, **Batch Gradient Descent** technically "converged" in only 2 epochs. However, the learning curves (Figure 2) reveal that this was a **False Convergence**.

- Because Batch GD updates weights only once per epoch, the reduction in loss per epoch is extremely slow and smooth.
- The change in loss was smaller than the threshold (10^{-4}) simply because the model was learning too slowly, not because it had reached the optimum.

5.2 Stochastic Noise and Thresholds

Conversely, efficient optimizers like **Adam** and **Momentum** show a dramatic reduction in loss in the first 5 epochs, functionally solving the classification problem. However, they are listed as "> 20 epochs" in the table.

- Since these were run with **Batch Size = 1** (SGD mode), the loss fluctuates slightly due to stochastic noise.
- Even when the model is highly accurate, the jitter in loss often exceeds 10^{-4} , preventing the strict stopping condition from triggering.

5.3 Impact of Network Depth

- **Adaptive Methods (Adam, RMSprop):** These optimizers proved robust against increased network depth. They maintained rapid convergence rates for 3, 4, and 5 layers.
- **Standard SGD:** Showed slower convergence and higher susceptibility to local minima as the network became deeper.

6 Conclusion

The experiment confirms that **Adam** and **RMSprop** are the most effective optimizers for this task, offering the fastest functional convergence. While **Batch Gradient Descent** provides a smooth trajectory, it is computationally inefficient and prone to premature stopping under strict threshold-based criteria. For practical implementations on MNIST, Adaptive methods are recommended.